

Student Meal Planner

Sprint Retrospectives

A Project Management Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

About These Retros

I ran informal sprint retrospectives at the end of each development phase. Since this is a solo project, the retros are just me reflecting on what happened, what went wrong, and what I'd change. They're short because there's no team to discuss with, but they helped me catch patterns I kept repeating.

[NOTE: Update dates and sprint numbers to match your actual sprint schedule. Add or remove sprints as needed.]

Sprint 1: Project Setup and Initial Pages

What went well: Got the React + Vite frontend running quickly. The Home page came together fast because I had a clear idea of what it should look like. Setting up the ASP.NET Core backend was straightforward once I figured out the connection string situation. Having a working frontend and backend talking to each other by the end of the sprint felt good.

What didn't go well: I used Tailwind for the Dashboard without thinking about whether I'd use it everywhere. That created an inconsistency problem that I had to fix later. Should've picked one CSS approach from the start. Also spent too long on visual polish before the core functionality was working.

What I'll change: Make architectural decisions (like CSS approach) before writing page code, not after. Focus on functionality first, then polish.

Sprint 2: More Pages and the Tailwind Problem

What went well: Built the Recipes page and got recipe cards rendering from local data. The card layout worked nicely with the dark theme. Started the Meal Planner page layout. Filter buttons for recipe categories were quick to implement.

What didn't go well: The Tailwind vs. custom CSS inconsistency became a real issue. Dashboard looked different from Recipes because they used different styling systems. Debugging meant checking two completely different places. I also ran into a bug where `recipe.name?.toLowerCase()` was crashing because some data used `title` instead of `name`. Inconsistent data models caused problems on the frontend.

What I'll change: Decided to remove Tailwind completely and standardize on custom CSS. Also realized I need a consistent data model (always `title`, never `name` for recipe titles).

Sprint 3: Tailwind Removal and Recipe View

What went well: Ripped out Tailwind entirely. Converted Dashboard and Meal Planner to custom CSS with the `ComponentName-element-subelement` naming convention. The JSX got way more readable. Built the Recipe View page from an HTML mockup. The ingredient checklist with `useState` worked on the first try. Wired up recipe card navigation so clicking a card actually goes to the detail page.

What didn't go well: An accidental `import RecipeView` from `"./RecipeViewPage.jsx"` in `HomePage.jsx` caused a 500 error that took me a bit to track down. The `HomePage.jsx` file also got truncated at some point (missing closing tags on `HomeCreateAccountTab`), which caused another confusing error. Both were silly mistakes that a linter should've caught.

What I'll change: Pay more attention to import statements. Consider adding ESLint to catch this stuff automatically. Also, test the full app after every major change instead of only testing the page I'm working on.

Sprint 4: Layered Architecture Migration

What went well: Split the backend from one flat project into four class library projects (Domain, DataAccess, Business, API). The separation made the codebase immediately easier to understand. Dependency injection worked cleanly once everything was registered in `Program.cs`. Writing the repository pattern with ADO.NET wasn't as hard as I expected.

What didn't go well: Namespace issues everywhere after moving files between projects. `using Grubs4Scrubs.Business` instead of `using Grubs4Scrubs.DataAccess` in `RecipeService.cs` took a while to spot. NuGet packages needed to be installed in the right project (Microsoft.Data.SqlClient in DataAccess, not API). Connection string problems kept coming back. A BOM character appeared in one of the files and caused a mysterious build error.

What I'll change: When restructuring projects, make a checklist: update namespaces, update project references, install NuGet packages in the correct project, test the build after each file move instead of moving everything at once and hoping it compiles.

Sprint 5: Documentation and Portfolio

What went well: Wrote three research documents (Layered Architecture, Frontend Design Decisions, Database & Data Access) using the DOT framework structure. Having clear research questions and sub-questions made the writing process faster than I expected. The documents also helped me articulate decisions I'd made instinctively but hadn't formally justified.

What didn't go well: Documentation took longer than planned because I kept going back to check implementation details I hadn't written down when I built them. The Target Audience Research and Competitor Analysis should've been done earlier in the semester when the analysis was fresh.

What I'll change: Write documentation closer to when the work happens. It's harder to remember details and reasoning weeks after the fact. Next time, at least jot down notes during development so the formal write-up is easier.

Sprint 6: Shopping List, Recipe Edit, and Authentication

What went well: Built the entire ShoppingItem backend (model, interface, repository, service, controller) and connected it to a fully functional React shopping list page with inline editing. The inline edit pattern (using an `editingId` state to toggle between display and input mode) worked cleanly. Added an edit modal to RecipeViewPage that handles JSON-to-text conversion for ingredients and instructions. Built the JWT authentication backend from scratch: AuthController with register and login endpoints, BCrypt password hashing, JWT token generation with claims and HMAC-SHA256 signing, and the middleware configuration in Program.cs.

What didn't go well: The auth system had a lot of small bugs during implementation. Wrong variable names (`existing` vs `user`), reversed if-checks (checking `== null` instead of `!= null`), route attribute typos (`{register}` instead of `register`), and autocorrect changing `validate` to `validated` in Program.cs. Each one was a 5-minute fix but they added up. The HomePage footer positioning is still broken because the page uses absolute positioning throughout, and a CSS fix for the footer broke every other page.

What I'll change: Test each method right after writing it instead of building the whole controller and then testing. The auth bugs would've been caught faster if I'd tested register before writing login. For CSS, don't use absolute positioning for page-level layout. It creates cascading problems that are hard to fix without a refactor.

[NOTE: Update with any other details from this sprint that I missed.]

Overall Patterns I Noticed

Looking back across all the sprints, a few things keep coming up:

I tend to make architectural decisions too late. The Tailwind mess, the flat project structure, and the inconsistent data models all happened because I started building before deciding on an approach. The fix is simple: spend thirty minutes planning before writing code.

Namespace and import errors are my most common bug. Every time I move or rename something, I forget to update at least one import or using statement. A linter or better IDE setup would catch most of these.

Documentation is easier when it's done soon after the work. The research docs I wrote right after implementing something were faster and more detailed than the ones I wrote from memory weeks later.